EXP - 13

Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Aim

To analyse the limitations of low-code platforms (with Zoho Creator as the reference platform) when applied to a complex enterprise scenario.

Introduction

Low-code platforms speed up development using visual builders, pre-built components and minimal hand-written code. They are excellent for forms, workflows, dashboards and departmental apps. However, large, highly complex enterprise systems expose constraints that every architect should understand before choosing the platform.

Sample Complex Enterprise Scenario

A multinational manufacturing company wants one system for: ERP-like inventory and procurement, real-time IoT machine data, a customer portal with millions of users, complex pricing engines, advanced analytics, strict regulatory audits and integration with several legacy systems.

Key Limitations

1. Scalability and Performance

- Platforms enforce limits on records, storage, API calls and script execution time/statements.
- Very high-volume or real-time workloads (millions of transactions per hour, streaming IoT data) may not suit a general low-code engine.
- Complex reports over huge datasets can become slow.

2. Customisation Constraints

- Visual builders cover common patterns; unusual UI/UX or algorithms may be impossible or require workarounds.
- Proprietary scripting languages (such as Deluge) are less powerful than general languages and have limited libraries.

3. Vendor Lock-in

- Applications are tied to the vendor's runtime, data model and scripting language.
- Migrating away usually means rebuilding, not exporting code.
- Pricing and feature roadmaps are controlled by the vendor.

4. Integration Complexity

- Legacy or custom protocols may need middleware or custom code.
- API rate limits and connector coverage affect integration design.

5. Complex Business Logic

- Deep recursion, heavy computation, machine learning pipelines or multi-step transactions with rollback are hard to model.
- Limited support for fine-grained transaction control compared with traditional back-end code.

6. Security and Compliance Control

- Security depends on the vendor's shared-responsibility model; organisations get less control over infrastructure, hosting and encryption internals.
- Some industries need on-premise hosting or specific certifications.

7. Testing, Debugging and DevOps

- Limited automated testing frameworks, version control and CI/CD compared with pro-code stacks.
- Debugging is mostly through logs and platform tools.

8. Governance and Shadow IT

- Easy app creation can lead to uncontrolled, duplicated or undocumented applications.

9. Cost at Scale

- Per-user licensing can become expensive for thousands of users.
- Additional cost for higher limits, environments and add-ons.

10. Skills and Maintainability

- Apps built quickly by non-experts may have poor data models, causing technical debt.

Comparison: Low-Code vs. Pro-Code

Aspect	**Low-Code**	**Pro-Code**
Speed to build	Very fast	Slower
Flexibility	Moderate	Very high
Scalability	Platform-limited	Architecture-dependent, high
Skill needed	Lower	High
Lock-in	High	Lower
Maintenance	Vendor handles infra	Team handles infra

Mitigation Strategies

- Use a hybrid approach: low-code for workflows/UI, pro-code (e.g., custom functions, Zoho Catalyst) for heavy logic.
- Keep core systems of record separate and integrate via APIs.
- Design the data model carefully before building.
- Establish a Centre of Excellence for governance.

- Evaluate limits (API, storage, execution) early with a proof of concept.
- Plan an exit strategy and export routines for data.

Case Study Conclusion (Sample)

For the manufacturing scenario, Zoho Creator is well suited to the approval workflows, service requests, dashboards and partner portals, but the high-volume IoT ingestion and complex pricing engine should be built with pro-code services and integrated. The best outcome comes from choosing the right tool for each layer.